IEM 4723 Information Systems Design

Dr. Paritosh Ramanan

School of Industrial Engineering and Management
Oklahoma State University

Fall 2026

Welcome to IEM 4723 Information Systems Design

- **Course Codes:** IEM 4723 (Undergraduate) / IEM 5723 (Graduate)
- **Semester:** Fall 2026
- **Classroom:** EN310
- **Lecture Time:** Tuesdays and Thursdays, 10:30am-11:45am
- **Instructor:** Dr. Paritosh Ramanan
- **Office:** 354 Engineering North
- **Email:** paritosh.ramanan@okstate.edu

Course Description - Part 1

- Design and implementation of information systems
- Git and GitHub for version control and coursework management
- Data storage using relational (RDBMS) and key-value databases
- Hands-on implementation and manipulation of databases
- Containerization with Docker for portable, reproducible applications

Course Description - Part 2

- Data visualization using Python and Streamlit
- Building interactive dashboards and web applications
- AI agent development with open-source frameworks
- LLM integrations and MCP (Model Context Protocol)
- Enterprise tools (ERP and GIS)

Course Topics (Tentative)

- **Introduction and Git Basics (Lec 1)** - Version control fundamentals
- **Git Basics Continued (Lec 2)** - Workflow and collaboration
- **Introduction to Containers (Lecs 3-6)** - Docker fundamentals
- **Entity-Relationship Modeling (Lec 7)** - Database design
- **Relational Database Systems (Lecs 8-13)** - RDBMS and SQL
- **Midterm Exam** - covering material through Lecture 13
- **NoSQL and Key-Value Storage (Lecs 14-15)** - Redis
- **Data Visualization with Streamlit (Lecs 16-20)** - App development
- **AI Computing Ecosystem (Lecs 21-24)** - LLMs and AI Agents
- **Enterprise Tools (Lec 25)** - ERP and GIS
- **Project Presentations (Lecs 27-28)**

Textbooks and Resources

- **[H]** Harrington, J.L., *Relational Database Design and Implementation*, 4th Edition
- **[D]** Iliev, B., *Introduction to Docker* (free eBook)
- **[S]** Streamlit Documentation and *30 Days of Streamlit* (free)
- **[R]** DigitalOcean, *How To Manage a Redis Database* (free eBook)
- **References:**
  - Valacich et al., *Essentials of Systems Analysis and Design*
  - Whitten & Bentley, *Systems Analysis and Design Methods*
  - Class Notes/handouts (Git and GitHub basics, AI Agent Fundamentals)

Prerequisites

- IEM 1412 or equivalent programming experience
- Basic understanding of computer systems
- Willingness to learn new technologies and tools

Technology Requirements

Students will need access to a computer for daily work, assignments, and projects. Various free and open-source tools will be used throughout the course.

Grading Policy

Assignment	15%
Midterm Exam	25%
Project	35%
Final Exam	25%

- Two exams will be given in this course
- No make-up exam without prior arrangements
- Exams are closed book and closed notes
- Calculator allowed, no electronic devices

Letter Grade Scale

- **A:** 90% and above
- **B:** 80% - 89.9%
- **C:** 70% - 79.9%

Class Policy

- Attendance is essential for success in this course
- Excessive absences may impact your final grade
- Students are responsible for all material covered in class
- Regular participation in discussions is expected
- Exams must be taken on scheduled dates
- Documentation required for make-up exams

Communication

All course communication will be through Canvas and official OSU email.

Student Support Resources

- Free resources available for financial, mental health, and suicide prevention support
- Visit: https://ssc.okstate.edu/
- Counseling and psychological services available
- Academic success resources through the university
- Disability services available for students with documented needs

Don't Hesitate to Seek Help

Whether it's academic or personal, the university has resources available to support your success and well-being.

Office Hours and Contact

- **Office:** 354 Engineering North
- **Email:** paritosh.ramanan@okstate.edu
- **Office Hours:** By appointment (send email to schedule)
- **Response Time:** I will strive to respond to emails within 24-48 hours
- **Canvas Inbox:** Use the Canvas messaging system for course-related questions

Getting Help

- Attend office hours or schedule an appointment
- Use Canvas Inbox for course questions
- Check announcements and discussions first
- Be specific in your questions

Why Git and GitHub?

- **Version Control:** Track changes to your code and documents over time
- **Collaboration:** Work with others on the same project
- **Backup:** Store your work in the cloud
- **Release Management:** Create and manage different versions of your work
- **Professional Skill:** Essential for modern software development

What is Git?

- Git is a **distributed version control system**
- All repositories are equivalent and contain the entire history
- Commits are local (don't need to be connected to a central repository)
- But a central repository is still useful for collaboration
- Created by Linus Torvalds in 2005 for Linux kernel development
- Open-source and free

Git Concepts

- **Working Copy:** Your actual files on disk
- **Staging Area (Index):** Files prepared for the next commit
- **Local Repository:** Your local history of commits
- **Remote Repository:** Repository on a server (e.g., GitHub)

Basic Git Commands

- `git init` - Initialize git for the current directory
- `git status` - Show status of files
- `git add <file>` - Stage a file for commit
- `git commit` - Commit staged files to local repo
- `git log` - Show commit history
- `git diff` - Show changes between working and repo
- `git clone` - Clone a remote repository
- `git push` - Push commits to remote repository
- `git pull` - Fetch and merge changes from remote
- `git fetch` - Fetch changes from remote without merging

GitHub Basics

- GitHub is a web-based hosting service for Git repositories
- Provides collaboration features, issue tracking, and more
- Free for public repositories
- Paid plans for private repositories (free for students)
- Used by millions of developers worldwide

Key GitHub Concepts

Repository (Repo): A project's container

Fork: A copy of someone else's repository

Clone: Download a repository to your local machine

Commit: A snapshot of your changes

Pull Request: Propose changes to a repository

First Time Git Setup

- Configure your identity (needed for all commits):
  - `git config --global user.name "Your Name"`
  - `git config --global user.email your@email.com`
- Configure your default editor:
  - `git config --global core.editor vim`
- Verify your configuration:
  - `git config --list`

Basic Git Workflow

1. Create or clone a repository
2. Make changes to files in your working directory
3. Stage the files you want to commit: `git add <file>`
4. Commit the staged changes: `git commit -m "message"`
5. Push your commits to the remote repository: `git push`

Command	Description
`git init`	Create new repository
`git add .`	Stage all files
`git commit -m "msg"`	Commit with message
`git push origin main`	Push to remote

Forking and Cloning

Forking a Repository:

- Click the "Fork" button on GitHub
- Creates a copy under your GitHub account
- Keep your fork synced with the original

Cloning a Repository:

- `git clone <repository-url>`
- Creates a local copy of the repository
- Sets up remote tracking automatically
- Example: `git clone https://github.com/user/repo.git`

The Fork-Clone-Edit-Push Cycle:

Fork (on GitHub) $\rightarrow$ Clone (to local) $\rightarrow$ Edit $\rightarrow$ Commit $\rightarrow$ Push $\rightarrow$ Pull Request

Branches (Introduction)

- Branches allow you to develop features independently
- Default branch is usually `main` or `master`
- Create a new branch: `git branch <branch-name>`
- Switch branches: `git checkout <branch-name>`
- Create and switch: `git checkout -b <branch-name>`
- Merge branches: `git merge <branch-name>`
- List branches: `git branch`

Best Practice

Create a new branch for each feature or fix. This keeps your main branch clean and makes it easier to manage changes.

Questions?

Thank You!

Let's get started with Git and GitHub!